

PROJECT REPORT

Shrenika Rajpoot

25BAI11384

—

AURA

(Advanced Universal Response & Assistance
System Java-based personal desktop assistant)

—

Course

Programming in Java — CSE2006

TITLE

AURA — Advanced Universal Response & Assistance System

Java-based personal desktop
assistant

Course: Programming in Java —
CSE2006

Technology: Java

Database: MySQL

Database Connectivity: JDBC

IDE: Visual Studio Code

AURA (Advanced Universal
Response & Assistance System) is
a Java-based personal desktop
assistant that seeks to facilitate
everyday computer tasks through
a command-driven interface.

The project focuses on applying
the concepts studied in the
Programming in Java course to a
single application. Instead of
implementing each Java concept
as a stand-alone demonstration
program, AURA applies concepts
such as OOP, exception handling,
multithreading, collections, file
handling, and JDBC to provide a
complete implementation.



Abstract

AURA enables a user to manage personal notes and tasks, perform calculations, check date and time, maintain command history, store user-related information, and maintain application logs using a database.

The project is designed in a modular fashion, enabling new commands and services to be added without having to alter the complete application. The project, therefore, provides a practical exposure to Java programming while also showing how different Java concepts can be used in a practical application.

Introduction

Modern computers are capable of performing a large number of tasks, but require a user to interact with different applications separately to carry out a simple activity. For example, a user may need one application for notes, one for tasks, a calculator for calculations, and another for managing stored information.

AURA is developed using the idea of combining some of these basic functions into a single Java application.

However, AURA draws inspiration from the concept of a personal digital assistant, and implements a realistic Java-based implementation that can be implemented and understood using the concepts covered. The project also helps to provide a practical understanding of the way Java concepts are used in a wider context.

Problem Statement

Users commonly perform small repetitive tasks such as creating notes, managing tasks, checking information, performing calculations, and reviewing previous activities. However, performing these operations using different applications can prove to be tedious.

Problem Statement:

To develop a modular Java-based personal assistant that can perform common user tasks through a unified interface while demonstrating core Java programming, OOP, exception handling, multithreading, collections, I/O Streams and JDBC.

Objectives

The objectives of AURA are:

- To implement a practical application using Java
- To apply object-oriented programming concepts in a real project
- To provide a single interface for performing common personal tasks
- To demonstrate proper exception handling and input validation
- To apply multithreading for background operations
- To apply Java Collections for managing application data
- To implement file and stream based input/output operations
- To implement JDBC for communication between Java and a relational database
- To maintain user information, tasks, notes and command history
- To create a modular application that can be further extended with additional features in the future

Scope of the Project

The initial scope of AURA includes:

- User Management
- User registration
- User login
- User information
- Assistant Commands
- Greeting and basic interaction
- Date and time
- Calculator
- Help command
- Exit command
- Task Management
- Add task
- View tasks
- Update task
- Delete task
- Mark task as completed
- Notes Management
- Create notes
- View notes
- Search notes
- Delete notes
- History & Logging
- Command history
- Application activity logs
- Error logs

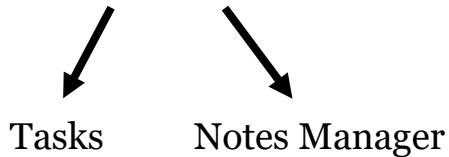
- Database
- User records
- Tasks
- Notes
- Command history
- Background Operations
- Reminder service
- Background logging
- Other independent tasks through threads

Proposed System

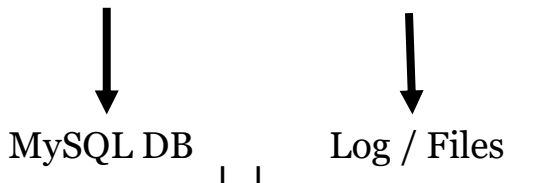
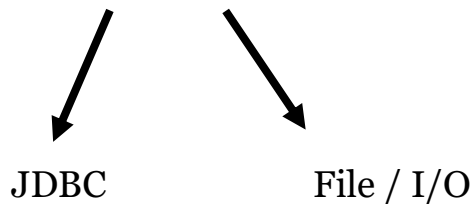
AURA INTERFACE



COMMAND PROCESSOR



SERVICE LAYER



Major Features

- Command Processing

AURA processes commands from the user, identifying the request being made.

Example:

User: add task Complete Java project

AURA: Task added successfully.

The command is processed from an interface and separated from the service for the actual operation to be performed.

- Task Management

Users can create and manage personal tasks.

Example:

add task Complete Java assignment

view tasks

complete task 3

delete task 3

Task information is persisted using the database.

- Notes Management

Users can create and retrieve notes.

Example:

add note Java supports object-oriented programming.

search note inheritance

view notes

This feature demonstrates both data management and collection processing.

- Calculator

AURA can perform basic mathematical operations on a user-provided input.

Example:

calculate 25 + 15

Output:

Result: 40

Input validation and exception handling is demonstrated, to prevent invalid operations from terminating the program.

- Date and Time

AURA can provide the current date and time, using Java's date and time functionality.

Example:

User: time

AURA: Current time: 10:30 PM

- Command History

AURA maintains a record of commands entered by the user.

This can be managed by using Java Collections, or by persisting the collections to databases for more persistence.

- Logging

Important application activities and errors are recorded in log files.

For example:

[INFO] User logged in

[INFO] Task created

[INFO] Note saved

[ERROR] Invalid command

This demonstrates practical use of Java I/O.

Object Oriented

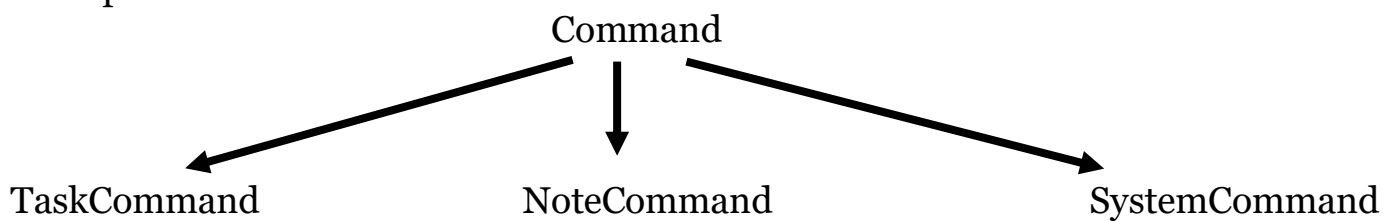
Object-oriented programming is used to support the basic architecture of AURA.

Possible core classes are:

- User
- Task
- Note
- Command
- Assistant
- TaskManager
- NoteManager
- CommandProcessor
- DatabaseManager
- FileManager
- ReminderService
- Encapsulation
- Class fields are kept private, and are accessed through appropriate methods.
private String taskName;
private boolean completed;
- Inheritance

Common functionality can be implemented using parent and child classes where appropriate.

For example:



- Polymorphism
Different command types are able to implement the same interface but provide their own implementation.
- Abstraction
Interfaces and abstract classes can be used to define common behavior, without having to expose unnecessary details.

Exception Handling

Exception handling is used to prevent the application from becoming unresponsive. Examples include:

InvalidCommandException
TaskNotFoundException
NoteNotFoundException
DatabaseConnectionException
InvalidInputException

The project demonstrates:

```

try
catch
finally
throw
  
```

throws

multiple catch blocks

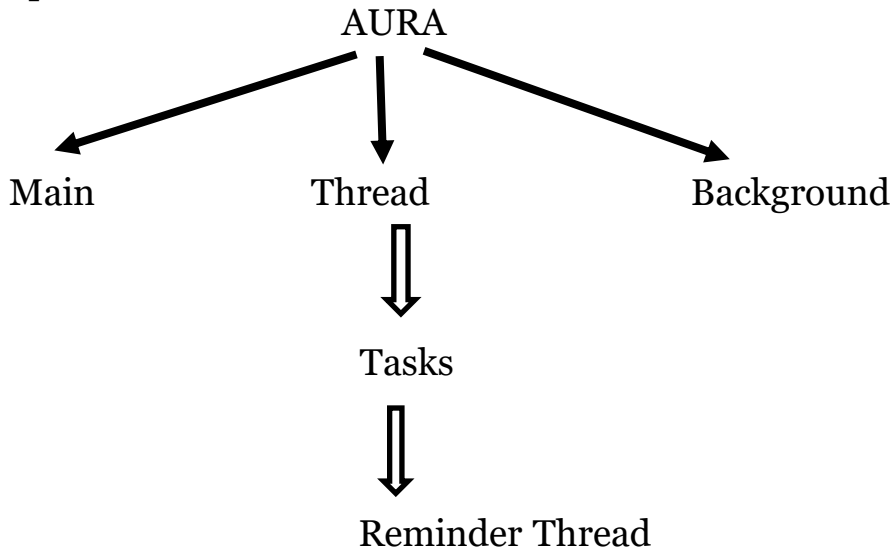
user-defined exceptions

Preventing the entire application from being terminated due to one erroneous input is demonstrated.

Multithreading

Threads can be used to separate operations that are run independently from the main command-processing flow.

For example:



Possible background operations include:

Reminder checking

Background logging

Periodic system checks

Java thread creation, lifecycle and synchronization are demonstrated, in cases where shared resources are to be used.

Java Collections Framework

Collections are used wherever dynamic data needs to be stored and processed.

Examples:

ArrayList

Used for maintaining lists such as tasks or notes at runtime.

ArrayList tasks;

HashMap

Can be used for command mapping.

HashMap commands;

Example:

"add task" → TaskCommand

"add note" → NoteCommand

"time" → TimeCommand

Stack

Can be used for maintaining recently executed commands or implementing command history.

File Handling and I/O Streams

AURA uses Java I/O for application logs, exports, and supporting files.

Technologies include:

File

FileReader

FileWriter

BufferedReader

BufferedWriter

InputStream

OutputStream

For example:

logs/

aura.log

error.log

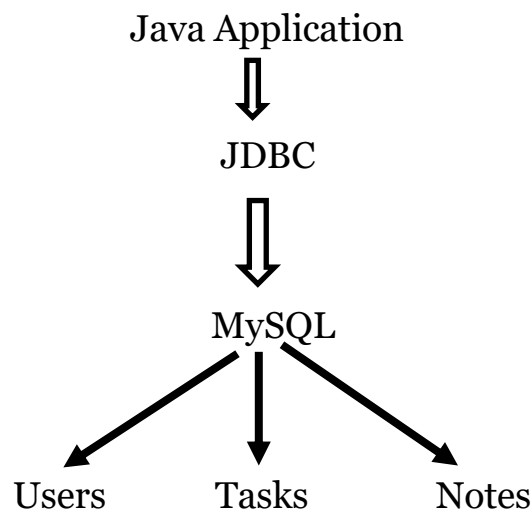
data/

exported_notes.txt

This provides a practical implementation of the I/O concepts covered in the syllabus.

Database and JDBC

AURA uses MySQL as its persistent database, and JDBC as the connection layer between Java and MySQL.



Possible tables:

users

tasks

notes

command_history

JDBC is used for:

Establishing database connections

Executing SQL queries

Inserting records

Retrieving records

Updating records

Deleting records

Prepared statements should be used when working with user-provided values.

Database Operations

Create (Adding a task):

INSERT INTO tasks (...) VALUES (...);

Read (Displaying tasks):

SELECT * FROM tasks;

Update (Completing a task):

UPDATE tasks SET completed = ? WHERE id = ?;

Delete (Deleting a task):

DELETE FROM tasks WHERE id = ?;

Therefore, AURA demonstrates complete CRUD operations through JDBC.

Technology Stack

Component

Technology

Programming Language

Java

Database

MySQL

Database Connectivity

JDBC

IDE

Visual Studio Code

Version Control

Git & GitHub

Data Structures

Java Collections Framework

File Processing

Java I/O Streams

Concurrency

Java Threads

Hardware & Software Requirements

Hardware

Computer/Laptop

Minimum 4 GB RAM

Minimum 500 MB free storage

Software

JDK 17 or later

Visual Studio Code

Java Extension Pack

MySQL Server

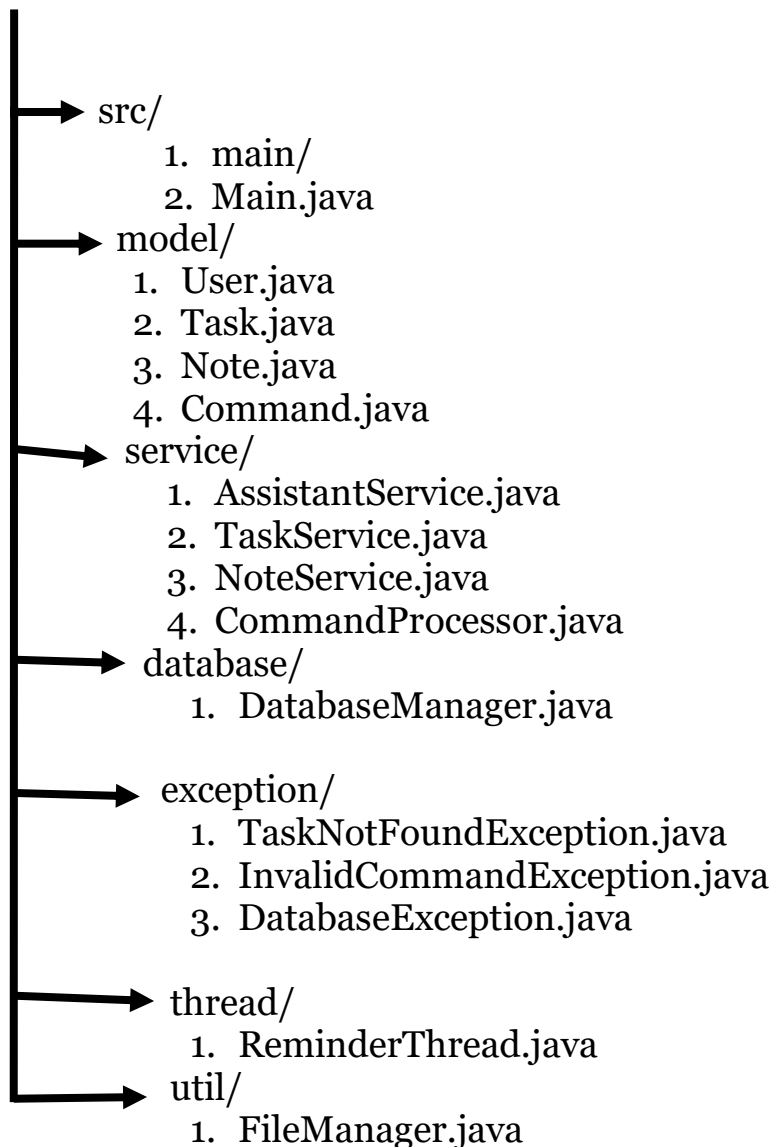
MySQL Connector/J

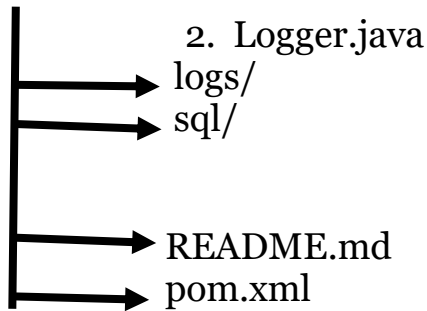
Git

Project Structure

A clean structure for AURA can be:

AURA





If we later use Maven, the structure can be adjusted to standard Maven conventions.

Mapping

AURA Implementation

Module 1 — Java Introduction & Flow Control

Input/output, variables, operators, conditions, loops, switch, break, continue

Module 2 — Java OOP

Classes, objects, constructors, access modifiers, this, super, inheritance, interfaces, polymorphism, encapsulation, enum

Module 3 — Exception Handling & Multithreading

Custom exceptions, try-catch, throw/throws, threads, thread lifecycle, synchronization

Module 4 — Java List & I/O Streams

ArrayList, HashMap, Stack, Strings, arrays, FileReader/FileWriter, BufferedReader/Writer, streams

Module 5 — Database Applications with JDBC

MySQL connection, JDBC, PreparedStatement, ResultSet, CRUD operations, database management

This is the important distinction:

AURA isn't five unrelated syllabus programs put together.

Each module contributes to the functioning of the same application.

Expected Results

After successful implementation, AURA is expected to:

Accept and process user commands.

Manage tasks and notes.

Store information persistently.

Handle invalid input without crashing.

Execute selected background operations using threads.

Maintain application logs.

Perform database CRUD operations using JDBC.

Demonstrate practical use of Java OOP and Collections.

Provide a foundation for adding more intelligent assistant features later.

Future Scope

AURA can be vastly expanded in the future.

Possible improvements include:

Voice Interaction

Users can interact with AURA by speaking instead of typing in commands.

"Hey AURA, add a task for tomorrow."

Text-to-Speech

AURA can speak back.

AI-Based Command Understanding

Instead of requiring fixed commands, AURA can understand various ways of saying the same thing.

GUI

A graphical interface can be developed using JavaFX.

Smart Reminders

The assistant can remind a user about upcoming tasks.

External APIs

Weather, news, calendar, or other external services can be integrated.

User Personalization

AURA can remember preferences of different users.

Conclusion

AURA demonstrates how the concepts from the Programming in Java course can be combined together to develop a practical software application. The project combines Java fundamentals, object-oriented programming concepts, exception handling, multithreading, collections, I/O Streams and JDBC within a modular system. The main learning outcome from the project isn't just to create a personal assistant, but understanding how individual concepts can come together to build a practical application. Using AURA, theoretical concepts can be connected with practical implementation, making the project to be both a useful application and a comprehensive demonstration of the concepts in the CSE2006 syllabus.